FULL STACK WEB DEVELOPMENT

An Internship Report Submitted to the Department of

INFORMATION TECHNOLOGY

AND

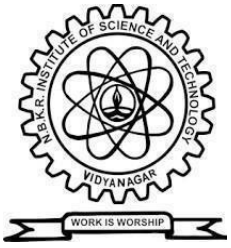
ARTIFICIAL INTELLIGENCE & DATA SCIENCE

By

Nikhil Dandolu (22KB1A1212)

Under The Esteemed Guidance Of

Slash Mark



DEPARTMENT OF IT AND AI&DS

N.B.K.R INSTITUTE OF SCIENCE AND TECHNOLOGY

(Autonomous)

VIDYANAGAR-524413, S P S R NELLORE DISTRICT, AP

2025-26

(DURATION: 16 WEEKS)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of project would be incomplete without the people who made it posable because of their constant guidance and encouragement crowned our efforts with success.

We convey our special thanks to **Dr. Y. Venkata Rami Reddy**, Chairman of **N.B.K.R. Institute of Science and Technology**, for providing excellent infrastructure in our campus for the completion of the internship.

We convey our special thanks to **Sri N. Ram Kumar Reddy**, Correspondent of **N.B.K.R. Institute of Science and Technology**, for providing excellent infrastructure in our campus for the completion of the internship.

We are grateful to **Dr. V. Vijaya Kumar Reddy, Director of N.B.K.R Institute of Science and Technology** for allowing us to avail all the facilities in the college.

We express our sincere gratitude to **Dr. A. Narayana Rao, Professor, Head of Department, IT and AI&DS** for providing exceptional facilities for successful completion of our internship.

We express our sincere gratitude to **Mrs. P. Chandrakala, Internship Associate, Assistant Professor, Department of IT and AI&DS**, for guiding us towards successful completion of internship.

We would like to convey our heartful thanks to **Staff members, Lab technicians and our friends**, who extended their cooperation in making this internship a successful one.

We would like to thank one and all who have helped us directly and indirectly to complete this internship successfully.

(INTERNSHIP ASSOCIATE)

Smt. P. Chandrakala

INDEX

List of the Concepts	Page No.
1. **HTML Fundamentals**	**01-02**
1.1 Basic Tags and Structure	01
1.2 Forms and Input Elements	01-02
1.3 Semantic HTML	02
2. **CSS Styling**	**03-04**
2.1 Selectors and Properties	03
2.2 Box Model and Positioning	03
2.3 Flexbox and Grid Layout	03-04
2.4 Transitions and Hover Effects	04
3. **Responsive Web Design**	**05-06**
3.1 Media Queries	05
3.2 Mobile-First Design	05
3.3 Fluid Layouts	05-06
4. **Image Editing for Web**	**07-08**
4.1 Cropping and Resizing	07
4.2 Compression and Optimization	07-08
4.3 File Formats (JPEG, PNG, SVG)	08
5. **UX/UI Design Basics**	**09-10**
5.1 Layout and Spacing	09
5.2 Color Schemes and Typography	09

List of the Concepts	Page No.

5.3 User-Centric Design Principles 10

6. **JavaScript Essentials** **11-12**

6.1 Variables, Data Types, Functions 11

6.2 DOM Manipulation 11-12

6.3 Event Handling 12

7. **ReactJS Basics** **13-15**

7.1 Components and Props 13

7.2 State Management 13-14

7.3 JSX Syntax 14-15

8. **API Integration** **16-18**

8.1 Fetch API and Axios 16

8.2 JSON Data Handling 17

8.3 Error Handling and Loading States 17-18

9. **Node.js Basics** **19-20**

9.1 What is Node.js? 19

9.2 Setting Up a Local Server 20

10. **PROJECTS** **21-51**

10.1 Fresh Mart Web Application 21-39

10.2 Portfolio Web Application 40-46

10.3 Ticket Billing Web Application 47-51

List of the Concepts	Page No.

11. **CERTIFICATE** **52-54**

11.1 Offer Letter 52

11.2 Certificate 53

12**. CONCLUSION** **54**

1. HTML FUNDEMENTALS

HTML (HyperText Markup Language) is the standard language used for creating and designing webpages. It provides the basic structure for web content and is considered the backbone of web development. HTML allows developers to structure text, images, multimedia, forms, and hyperlinks in a meaningful way that can be interpreted and displayed by web browsers.

Being a markup language, HTML uses a system of tags to define and organize content. These tags are enclosed within angle brackets and usually come in pairs, with an opening and a closing tag. HTML documents are the starting point of any website, and their structure ensures consistency, usability, and clarity across different platforms and devices.

1.1 Basic Tags and Structure

The basic structure of an HTML document is essential for defining how the content appears on the web page. Every HTML document begins with a declaration that defines the version of HTML being used. The root element <html> encompasses the entire document, which is then divided into two main sections:
<head> and <body>.

The <head> section contains information not directly visible on the webpage, such as the title, metadata, links to stylesheets, and scripts. The <body> section includes the actual content that is rendered on the screen, such as headings, paragraphs, links, images, and lists.

Some commonly used tags in HTML include:

- Headings (<h1> to <h6>) which define the importance of text.
- Paragraphs (<p>) used to display blocks of text.
- Line breaks (
) to shift content to a newline.
- Links (<a>) to navigate between web pages.
- Images (<img>) to display pictures or graphics.

Using these tags correctly helps ensure that the webpage is both functional and accessible.

1.2 Forms and Input Elements

Forms are a vital part of HTML that allow users to interact with a website by entering data. They are commonly used for user registrations, login pages, search bars, surveys, and contact forms. The <form> element serves as a container for input fields and is used to collect data from the user.

Forms can include a wide variety of input elements such as:

- Text fields for entering single-line or multi-line text.

- Email and password fields for user credentials.

    - Radio buttons and checkboxes for selecting one or more options.
    - Dropdown menus for choosing from a list.
    - Date pickers, number selectors, file upload fields, and more.

Each form element comes with attributes like name, value, required, placeholder, etc., which control its behaviour and interaction with the user. Additionally, forms use the action and method attributes to specify where and how the data should be submitted.

Forms play an important role in dynamic websites by allowing the website to collect and process user input efficiently.

1.3 Semantic HTML

Semantic HTML refers to the use of HTML tags that clearly describe their meaning and purpose within the webpage. Unlike generic tags like and , semantic tags provide context about the content they contain, making it easier for developers, search engines, and assistive technologies to understand the page structure.

Examples of semantic tags include:

- <header> – Defines the introductory content or navigation links.
- <footer> – Represents the bottom section of the page, often containing copyright or contact info.
- <article> – Specifies independent, self-contained content such as blog posts or news articles.
- <section> – Groups related content within a document.
- <nav> – Contains the navigation links.
- <aside> – Represents content that is related to the main content but can stand apart, like a sidebar.

The use of semantic HTML improves the accessibility and SEO (Search Engine Optimization) of a website. It also leads to more readable and maintainable code by clearly outlining the layout and purpose of each section of the webpage.

2. CSS STYLING

CSS (Cascading Style Sheets) is used to control the appearance and layout of HTML elements on a web page. It separates the structure of content (HTML) from the design and presentation. CSS allows web developers to apply styles such as colors, fonts, spacing, and positioning, making websites more visually appealing and user-friendly. It supports responsive design, ensuring websites look good on various devices and screen sizes.

2.1 Selectors and Properties

Selectors in CSS are used to target HTML elements that need styling. They define which elements the styles should apply to, while properties define how those elements should be styled. Common types of selectors include element selectors, class selectors, ID selectors, and group selectors. Pseudo-class selectors are also used to apply styles to elements in a particular state, such as when a user hovers over a button.

Properties are used to specify what kind of style will be applied to the selected elements. Examples include color, font-size, margin, padding, background-color, and border. By combining selectors with properties and values, CSS gives full control over the design and appearance of web pages.

2.2 Box Model and Positioning

The box model is a concept that describes how each HTML element is represented as a rectangular box. It includes four areas: content, padding, border, and margin. The content is the actual element, padding is the space around the content, the border wraps around the padding, and the margin is the outermost space separating the element from others.

Positioning in CSS defines how elements are placed on a page. There are several positioning methods: static, relative, absolute, fixed, and sticky. Static is the default flow of the document. Relative moves the element relative to its normal position. Absolute positions the element relative to its nearest positioned ancestor. Fixed positions the element relative to the browser window, and sticky allows the element to stick within a scrollable area. Proper use of positioning and the box model is essential for building organized and responsive layouts.

2.3 Flexbox and Grid Layout

Flexbox and Grid are powerful layout systems in CSS that help arrange elements efficiently on the web page.Flexbox is a one-dimensional layout model, useful for aligning items in a row or column.

It is ideal for distributing space and aligning content within a container, especially when the size of the items is unknown or dynamic.

Grid layout is a two-dimensional system used to design web pages using rows and columns. It provides more control over complex layouts and is suitable for creating large-scale designs like dashboards, galleries, and web templates. Grid allows developers to create responsive layouts without relying heavily on media queries.

Both Flexbox and Grid make it easier to create modern, clean, and flexible designs that adapt well to different screen sizes and devices.

2.4 Transitions and Hover Effects

CSS transitions allow smooth changes to property values over a specific duration, making websites look more dynamic and professional. Instead of changes happening instantly, transitions make them occur gradually, improving visual appeal.

Hover effects are triggered when the user moves their mouse over an element. They are commonly used to enhance interactivity, such as changing the background color of a button, increasing the size of an image, or displaying hidden content. These effects are useful for drawing attention to elements and providing feedback to user actions.

Using transitions and hover effects effectively can greatly enhance user experience and make a website feel more polished and responsive.

3. RESPONSIVE WEB DESIGN

Responsive Web Design is an approach to web development that ensures a website looks and functions well across all devices and screen sizes. This includes desktops, laptops, tablets, and smartphones. The main goal of responsive design is to provide an optimal user experience regardless of the device being used. It uses flexible layouts, scalable images, and CSS techniques to adapt the website's layout dynamically.

Responsive design improves usability, accessibility, and search engine rankings. It eliminates the need for separate websites for mobile and desktop users, making website maintenance easier and more efficient.

3.1 Media Queries

Media queries are a key feature of CSS used in responsive web design. They allow developers to apply different styles based on the characteristics of the device, such as screen width, height, resolution, or orientation. With media queries, a website can display content differently on small mobile screens and large desktop monitors.

For example, developers can use media queries to change font sizes, hide or show elements, rearrange layout structures, and adjust margins and paddings for various screen sizes. This helps in creating a seamless and consistent experience across different devices.

3.2 Mobile-First Design

Mobile-First Design is a development strategy where the design process starts with the smallest screen size (mobile devices) and then gradually adapts to larger screens. This approach ensures that the website is fully functional and user-friendly on mobile devices, which are widely used today.

By focusing on essential content and features first, developers can avoid clutter and prioritize usability. Once the mobile version is complete, styles and features are progressively enhanced using media queries to support tablets and desktops. This method improves performance and ensures that the most important information is always accessible.

3.3 Fluid Layouts

Fluid layouts use relative units like percentages instead of fixed units like pixels to define the width and height of elements. This allows the layout to expand or contract based on the size of the screen, making it flexible and adaptable to different resolutions.

Unlike fixed layouts that can break or require horizontal scrolling on smaller devices, fluid layouts adjust automatically to fit the available space. They work well in combination with media queries and flexible images to create a responsive design. Fluid layouts help maintain consistency and readability across various screen sizes, enhancing the overall user experience.

4. IMAGE EDITING FOR WEB

Images are an essential component of web design. They not only enhance the visual appearance of a website but also help in conveying messages and attracting users' attention. However, unoptimized or poorly edited images can significantly slow down a website, increase bounce rates, and negatively affect SEO rankings. Therefore, image editing for the web involves carefully preparing images to ensure they are visually appealing, appropriately sized, and optimized for fast loading.

Image editing includes tasks such as cropping, resizing, compressing, choosing the correct format, and optimizing image quality. These practices help in maintaining a balance between quality and performance, which is crucial for delivering a smooth and responsive user experience across all devices.

4.1 Cropping and Resizing

Cropping and resizing are basic but important techniques in image editing, especially for the web.

Cropping refers to trimming unnecessary or unwanted portions of an image to focus on the main subject. This technique helps in improving image composition and fitting the image into a specific design layout. Cropping can also help remove distracting elements from the background and highlight important content, which enhances the user's attention on key areas.

Resizing involves changing the dimensions of an image to make it suitable for various screen sizes or layout sections. For example, a high-resolution image taken from a camera may be too large for a website and must be resized to avoid slow page loading. Resizing ensures that images load quickly while still maintaining clarity and fitting well into the design structure. It also plays a significant role in responsive web design by allowing images to adapt properly to different screen sizes without loss of quality.

4.2 Compression and Optimization

Image compression and optimization are critical steps in preparing images for the web. They help reduce file sizes, which leads to faster website performance and better user experience, especially on mobile devices with slower internet connections.

Compression can be either **lossy** or **lossless**. Lossy compression removes some data from the image to reduce file size significantly but may slightly affect image quality. It is commonly used for photographs and detailed images. Lossless compression, on the other hand, reduces the file

size without any loss in quality, making it suitable for icons, logos, or graphics where clarity is important.

Optimization involves choosing the correct balance between image quality and file size. It includes removing unnecessary metadata, adjusting image resolution, and using tools or software that automatically compress images for web use. Optimized images improve SEO, reduce server load, and provide a smoother browsing experience for users.

4.3 File Formats (JPEG, PNG, SVG)

Choosing the appropriate image file format is essential for web performance and visual quality. Different formats are suited for different types of images based on their content, transparency, scalability, and required quality.

- **JPEG (Joint Photographic Experts Group)** is a widely used format for photos and images with many colors and gradients. It supports lossy compression, which means it reduces file size while slightly affecting quality. JPEGs are ideal for web photos, banners, and blog images.
- **PNG (Portable Network Graphics)** supports lossless compression and transparency. It is perfect for images requiring high quality and transparent backgrounds, such as logos, icons, or images with sharp edges and text.
- **SVG (Scalable Vector Graphics)** is a vector-based format used for logos, icons, and illustrations. It allows infinite scaling without losing quality, making it ideal for responsive design. SVG files are lightweight and can be styled and animated using CSS and JavaScript, which enhances flexibility in web development.

Using the right format ensures that images are displayed correctly and efficiently, contributing to both the design and performance of a website.

5. UX/UI DESIGN BASICS

UX (User Experience) and UI (User Interface) design are fundamental aspects of creating effective, user-friendly, and visually appealing websites or applications. While UI focuses on the visual layout and interactive elements of a website, UX emphasizes the overall experience a user has while navigating and interacting with it. Both work together to ensure that a website is not only attractive but also functional and easy to use.

Good UX/UI design improves user satisfaction, increases engagement, and helps achieve business goals by making websites more accessible, intuitive, and enjoyable. Elements such as layout, spacing, color, typography, and user-centric principles play a vital role in this process.

5.1 Layout and Spacing

The layout of a website refers to the way content is arranged on a page. It helps guide users' attention, ensures readability, and creates a logical flow of information. A well-structured layout ensures that users can easily find what they are looking for without confusion. Common layout patterns include grids, cards, and column-based designs.

Spacing is equally important in web design. Proper use of white space (or negative space) between elements makes a website look clean, organized, and easier to navigate. It improves visual hierarchy and allows users to focus on key content. Adequate spacing around buttons, images, and text also enhances touch usability, especially on mobile devices. Poorly spaced layouts can appear cluttered and reduce the overall quality of the user experience.

5.2 Color Schemes and Typography

Color schemes play a major role in setting the mood, tone, and brand identity of a website. The right choice of colors can attract attention, evoke emotions, and guide user actions. Color schemes should be consistent throughout the website to maintain visual harmony. Designers often use color theory and tools like the color wheel to select complementary or analogous color palettes.

In addition, color contrast is crucial for readability and accessibility. Using high-contrast text against background colors ensures that content is visible to all users, including those with visual impairments.

Typography involves the selection and arrangement of fonts on a website. It impacts how content is perceived and how easily it can be read. Good typography involves choosing the right font styles, sizes, and line spacing. It should maintain a balance between aesthetics and legibility.

Headings, subheadings, and body text should be clearly distinguishable, and font usage should be consistent to enhance readability and user experience.

5.3 User-Centric Design Principles

User-Centric Design is an approach that focuses on the needs, preferences, and behaviors of the end-users throughout the design process. It aims to create websites that are intuitive, useful, and tailored to meet user expectations.

Some key principles include:

- **Simplicity**: Keeping the design simple and uncluttered helps users focus on the main tasks without distraction.
- **Consistency**: Using uniform layouts, colors, and navigation across all pages builds user familiarity and trust.
- **Feedback**: Providing immediate visual or interactive feedback (like button highlights or confirmation messages) lets users know their actions are registered.
- **Accessibility**: Designing for all users, including those with disabilities, ensures inclusivity. This includes proper color contrast, keyboard navigation, and screen reader support.
- **Navigation**: Clear and predictable navigation helps users find their way without frustration, improving overall usability.

By applying user-centric design principles, developers can create more engaging and effective websites that meet the real needs of their target audience.

6. JAVASCRIPT ESSENTIALS

JavaScript is one of the core technologies of web development, enabling dynamic behavior and interactive features on websites. It allows developers to manipulate web pages in real-time, making the browsing experience more engaging. Mastering JavaScript is essential for building modern, user-friendly websites and applications.

6.1 Variables, Data Types, Functions

Variables are used to store values that can be referenced and manipulated later in a program. In JavaScript, variables are declared using different keywords, such as var, let, and const. Each keyword has its scope and behavior. var is function-scoped, while let and const are block-scoped. The key difference between let and const is that the value assigned to a variable declared with const cannot be changed, while let allows reassignment.

Data types in JavaScript are the classification of values. The primary data types include:

- **Primitive types**: These are simple values, such as strings, numbers, booleans (true or false), undefined, and null. They are immutable, meaning their values cannot be changed once assigned.
- **Non-primitive types**: These include objects, arrays, and functions. These types are more complex and can store multiple values, and their properties or values can be modified dynamically.

A **function** in JavaScript is a block of code designed to perform a specific task. Functions allow developers to create reusable code, which makes programs more efficient and easier to maintain. Functions can take inputs (called parameters) and return outputs (values). By organizing code into functions, developers can better manage the logic and structure of their programs.

6.2 DOM Manipulation

The **Document Object Model (DOM)** is an interface that allows JavaScript to access and manipulate the content and structure of web pages. The DOM represents the structure of an HTML document as a tree of objects, where each element, attribute, and piece of text in the HTML document becomes a node in this tree. JavaScript can interact with the DOM to modify the web page dynamically.

DOM manipulation is a critical skill in JavaScript, as it allows developers to change the content and structure of a webpage based on user interactions or other events. For example, JavaScript can be used to add new elements to the page, modify existing content, or remove

elements that are no longer needed. It also allows developers to modify the attributes and styles of elements, such as changing text colors, updating form fields, or altering layout properties like width and height.

By manipulating the DOM, developers can create interactive web pages, where the content is updated in real time without requiring a page reload. This dynamic capability is fundamental for building modern websites that offer an engaging user experience.

6.3 Event Handling

Event handling in JavaScript allows developers to respond to user actions, such as clicks, typing, mouse movements, and other interactions. Events are central to creating interactive websites and applications, as they trigger specific functions or actions when users interact with elements on the page.

In JavaScript, **event handlers** are used to detect and respond to various events. These events can be triggered by user interactions such as clicking a button, submitting a form, or hovering over an image. Event handling allows developers to create interactive elements, such as dropdown menus, modals, image galleries, and more.

For example, developers can use event handling to validate form submissions, display alerts when users click buttons, or dynamically update content based on user input. Event delegation, a common technique, involves assigning a single event listener to a parent element instead of multiple listeners for each child element, which can improve performance, especially on large websites with many interactive elements.

By utilizing event handling in JavaScript, developers can build rich, interactive websites that react instantly to user actions, providing a more engaging and intuitive experience.

7. ReactJS BASICS

ReactJS is an open-source JavaScript library that enables developers to build user interfaces, especially for single-page applications. React focuses on the development of reusable UI components that help in making dynamic applications. Unlike traditional web development that requires frequent reloading of the page, React uses a virtual DOM to efficiently update and render only the parts of the user interface that have changed, leading to faster performance and a better user experience. React's declarative syntax, component-based architecture, and powerful state management tools make it one of the most popular choices for building modern web applications.

7.1 Components and Props

Components are the foundation of React. They are self-contained, reusable pieces of UI that encapsulate their structure, styling, and behavior. React components break down the user interface into small, manageable pieces, each with its own functionality and rendering logic. Components can be either **functional** or **class-based**:

- **Functional Components**: These are simpler, stateless components written as functions. They receive input through **props** and render UI based on that input. With the advent of React hooks, functional components can now also handle local state and lifecycle methods.
- **Class Components**: These are more complex components that are defined as ES6 classes. They have additional features like local state and lifecycle methods, although functional components with hooks now often replace them.

In React, **Props** (short for properties) are how data is passed between components. A parent component can pass data to a child component via props, allowing for component reusability and flexibility. Props are **immutable**, meaning that they cannot be altered by the child component once passed, ensuring that data flows in one direction—from parent to child. This one-way data flow makes React applications more predictable and easier to debug.

Props allow components to be highly dynamic and customizable. For instance, a generic component can be reused multiple times, each time with different props, to render various types of content or behavior. By using props effectively, React allows developers to create reusable, modular components that can be shared and maintained across large applications.

7.2 State Management

In React, **state** refers to data that is specific to a component and can change over time.

Unlike props, which are passed down from a parent component, state is managed within the component itself. Each time the state of a component changes, React automatically re-renders the component and updates the UI to reflect the new state. This makes state an essential part of React applications, as it is used to control dynamic content, user input, and interaction.

State can be thought of as the "memory" of a component—it stores information that may change during the lifecycle of the component. For example, a form field may have a state variable that tracks the user's input, or a button may have a state that tracks whether it has been clicked.

React provides two main ways to manage state:

- **Local State**: This is the state that is specific to a single component. It is usually used for simple scenarios, like tracking whether a user has clicked a button or entering text in an input field.
- **Global State**: In larger applications, state may need to be shared across multiple components. React provides several options for handling this, including the **Context API** and state management libraries like **Redux**. These tools allow developers to manage and share state across the entire application, making it easier to manage complex, global data like user authentication status, theme settings, or shopping cart contents.

State management is crucial because it ensures that the application responds dynamically to user interactions, and helps maintain synchronization between the data and the UI. React's virtual DOM efficiently re-renders only the parts of the UI that are affected by state changes, which significantly improves performance.

7.3 JSX Syntax

JSX (JavaScript XML) is a syntax extension for JavaScript that allows developers to write HTML- like code directly within JavaScript. JSX is used extensively in React to define what the UI should look like. It combines the power of JavaScript with the simplicity of HTML-like syntax, making it easier to create and understand React components.

JSX may look similar to HTML, but it is actually syntactic sugar for React.createElement() calls. Although you can write JavaScript code in JSX, there are certain rules to follow, such as:

- Elements must be wrapped in a single parent element. If there are multiple elements, they need to be enclosed in a single parent tag, or a **Fragment** (a special wrapper tag that does not render anything in the DOM).

- JSX requires that all tags be properly closed. For example, self-closing tags like <input /> or <img /> must include a slash at the end.

.

JSX also promotes better readability and maintainability by enabling developers to define the structure and logic of a component in one place. The syntax of JSX feels more natural compared to writing JavaScript functions that generate HTML elements manually. This makes React components more intuitive and easier to work with, especially for teams working on large-scale applications.

8. API INTEGRATION

API (Application Programming Interface) integration plays a crucial role in modern web development. APIs allow different software systems to communicate with each other, enabling web applications to fetch, send, and display data dynamically. In React applications, integrating APIs allows developers to retrieve data from external services and display it in real-time, enhancing user experiences. This section explores the basics of working with APIs in React using the **Fetch API** and **Axios**, along with handling data and errors efficiently.

8.1 Fetch API and Axios

The **Fetch API** and **Axios** are two popular tools for making HTTP requests in modern web development. These tools allow developers to interact with external services, retrieve data, and send information to the backend server, making dynamic content possible.

- **Fetch API**: The Fetch API is a built-in JavaScript function that allows developers to make HTTP requests. It is easy to use and returns a promise, making it compatible with asynchronous operations. Fetch is widely used because it is native to JavaScript and does not require external libraries. By using fetch(), developers can make GET, POST, PUT, DELETE, and other types of HTTP requests to retrieve or send data to a server.
    - One of the advantages of the Fetch API is its flexibility. It allows for custom headers, error handling, and even streaming data. However, handling complex requests like file uploads and cancellations requires additional configuration.
- **Axios**: Axios is a popular third-party library for making HTTP requests. It simplifies the process of making requests by automatically transforming the response into JSON, managing request headers, and handling timeouts and cancellations. Axios supports both promises and async/await syntax and provides additional features like request and response interceptors, which allow developers to modify requests or responses globally before they are handled.
    - Axios is often preferred for its ease of use and rich feature set, especially in complex applications that require enhanced request management and customization.

Both Fetch and Axios enable developers to fetch external data from RESTful APIs or other web services, and both support modern JavaScript techniques like promises and async/await. The choice between Fetch and Axios generally comes down to the complexity of the project and developer preference, as both are widely used in the React community.

8.2 JSON Data Handling

When integrating APIs, the data returned is usually in the **JSON (JavaScript Object Notation)** format, which is lightweight and easy to parse. Handling JSON data is a crucial skill in API integration, as it is the standard format used for communication between clients and servers.

After an API request is made using Fetch or Axios, the response data is typically in the form of a JSON object. In JavaScript, the data needs to be parsed into a JavaScript object before it can be used. In the Fetch API, the .json() method is used to parse the JSON response, while in Axios, the response is automatically parsed into a JavaScript object.

Once the data is in a usable format, it can be displayed or processed within the React application. For example, you might display a list of items fetched from an API, update the state of a component based on the response data, or use the data in calculations.

Handling JSON data often involves iterating over the data and extracting specific values, especially when dealing with arrays or nested structures. Proper management of JSON data ensures that the web application remains responsive and user-friendly, with accurate and up-to-date information being displayed in the UI.

8.3 Error Handling and Loading States

When working with APIs, error handling and managing loading states are essential for providing a smooth user experience. APIs may fail for a variety of reasons—such as network issues, server errors, or invalid requests—so handling these errors gracefully is crucial to preventing app crashes and ensuring that users are informed of issues.

- **Error Handling**: In both the Fetch API and Axios, errors are handled using try-catch blocks (for async/await) or .catch() methods (for promises). Proper error handling can help developers identify when requests fail and provide fallback behavior, such as showing an error message or retrying the request. Common error scenarios include network failures, invalid responses, or authentication issues.
    - For example, when an API request fails, developers can display an error message to users, letting them know something went wrong, or attempt to recover from the error by retrying the request.
- **Loading States**: While fetching data from an API, there can be a delay between the request and the actual response. During this time, it is essential to show a **loading state** so that the user is aware that the content is being fetched and the application is not frozen. React

provides various ways to handle loading states, such as using state variables to track whether the data is still being fetched.

- A common approach is to display a spinner, loading text, or placeholder content until the data has been fully loaded. Once the data is fetched and the state is updated, the UI can be updated to show the actual content.

Effective error handling and loading state management are key to providing a responsive, user- friendly web experience. By anticipating potential errors and providing feedback during loading, developers can improve the reliability and usability of their applications.

9. Node.js BASICS

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 JavaScript engine. It enables developers to run JavaScript on the server side, which is typically used for client- side scripting in web browsers. By enabling JavaScript on the server side, Node.js allows for the development of scalable, fast, and efficient web applications, especially suited for I/O-heavy applications, real-time applications, and microservices.

Node.js has become one of the most popular tools in web development due to its non-blocking, event-driven architecture, which allows it to handle multiple concurrent connections efficiently. It has gained immense popularity as a server-side technology due to its scalability, speed, and flexibility.

9.1 What is Node.js?

Node.js is a runtime environment and framework that allows developers to write server-side applications using JavaScript. Traditionally, JavaScript was confined to the browser to handle client-side operations such as user interactions and content display. However, Node.js extends JavaScript beyond the browser, enabling the development of server-side applications.

One of the core features of Node.js is its non-blocking, event-driven architecture, making it an ideal solution for I/O-intensive applications. It can handle multiple tasks simultaneously by using a single-threaded event loop, which eliminates the need for multiple threads, making it resource- efficient. This is particularly important for applications that handle a large number of simultaneous connections, such as real-time chat apps, gaming servers, and APIs for web and mobile applications.

Node.js provides a vast ecosystem of open-source libraries and modules available via the Node Package Manager (NPM), which makes it easy to add functionality to applications without having to write everything from scratch. It also allows developers to use JavaScript throughout the entire development stack, from the client side (using libraries such as React or Angular) to the server side, offering a unified and efficient development experience.

Additionally, Node.js supports various modern programming features like asynchronous programming with callbacks, promises, and async/await, which enables developers to handle time-consuming tasks (like file reading, database queries, or HTTP requests) without blocking the main thread, ensuring better performance.

9.2 Setting Up a Local Server

Setting up a local server with Node.js is one of the first and fundamental steps in creating a Node.js application. This step helps developers understand how web servers work and allows them to build web applications that can handle requests, interact with databases, and serve dynamic content. Setting up a local server also allows for testing, debugging, and developing applications before deploying them to a production environment.

To set up a local server, the first step is installing Node.js on the machine. After installation, developers can begin creating a simple server to handle HTTP requests and provide responses. A server typically listens for incoming requests from clients (such as web browsers) and responds with appropriate content, like web pages, images, or data. Node.js comes with an in-built HTTP module that allows for the creation of basic web servers. However, many developers prefer using frameworks like Express.js to simplify this process and streamline the development of more complex applications.

When building a local server, it's common to configure the server to listen on a specific port (such as port 3000). This allows developers to visit their server through a web browser or API client by navigating to http://localhost:3000 (or the chosen port number). Once the server is set up, it can handle different types of requests, including GET requests to retrieve data or POST requests to send data to the server.

Additionally, creating a local server allows developers to implement essential features like routing (to map different endpoints to specific functions or views), middleware (to process requests before sending a response), and integration with databases to store and retrieve data. For example, developers might add functionality to the server to handle user authentication, serve dynamic content, or interact with a database to store user data or fetch records from a collection.

While developing locally, developers can test the server in real-time, allowing for rapid prototyping, debugging, and testing of features as they build the application. This local environment acts as a sandbox where developers can experiment with new ideas, troubleshoot errors, and ensure that their code is functioning as expected before deploying to a production server.

10. PROJECTS

10.1 Fresh Mart Web Application

Problem Statement: Design and develop a responsive product landing page for the **FRESH MART** web application that show cases an online grocery shopping experience. The page should include a navigation bar, a visually appealing hero section with product highlights and a call-to-action button, well-structured product categories. It must also provide detailed descriptions of available items,a pricing section displaying offers and discounts, and customer testimonials to build trust.Content should be organized using a clean column based layout for better readability and user experience. Additionally, optimize images through cropping and re-sizing to ensure fast loading and high visual quality across devices.

Abstract

The Fresh Mart web application is a modern product-focused platform designed to deliver a seamless and engaging online shopping experience for users. Built with a clean and structured layout, the application emphasizes visual appeal and usability by organizing content into well-defined sections using CSS. The homepage effectively showcases products with clear images, pricing details, and intuitive navigation, allowing users to explore items effortlessly. The design ensures that key elements such as featured products, categories, and offers are prominently displayed to capture user attention and encourage interaction

A major aspect of the Fresh Mart application is its focus on optimized visual content and performance. Product images are carefully selected, re-sized, and optimized to maintain high quality while ensuring fast loading times, which is essential for retaining user interest. The application also highlights important information such as product details, availability, and user-friendly browsing options, enhancing the overall shopping experience. This balance between aesthetics and performance contributes to a smooth and efficient interface that meets user expectations.

Furthermore, the application is developed with responsiveness as a core principle, ensuring compatibility across desktops, tablets, and mobile devices. Using CSS-based responsive design techniques, the layout adapts dynamically to different screen sizes, providing a consistent and accessible experience for all users. By combining effective design, optimized media, and user-centric features, the Fresh Mart web application successfully creates an interactive platform that not only showcases products but also drives user engagement and supports business objectives such as conversions and customer satisfaction

Source code :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>FreshMart - Your Neighborhood Grocery Store</title>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&family=Montserrat:wgh
t@700;800;900&display=swap" rel="stylesheet">
  <style>
    :root {
      --primary: #4caf50;
      --primary-light: #80e27e;
      --primary-dark: #087f23;
      --secondary: #ff9800;
      --secondary-light: #ffc947;
      --secondary-dark: #c66900;
      --accent: #ff5722;
      --accent-light: #ff8a50;
      --bg: #f8fff8;
      --bg-light: #ffffff;
      --text: #222;
      --text-light: #555;
      --card-bg: #ffffff;
      --shadow: 0 8px 20px rgba(0,0,0,0.08);
      --shadow-hover: 0 15px 30px rgba(0,0,0,0.15);
      --transition: all 0.3s ease;
      --border-radius: 15px;
      --gradient: linear-gradient(135deg, #4caf50, #8bc34a);
      --gradient-secondary: linear-gradient(135deg, #ff9800, #ffc947);
      --glass-bg: rgba(255, 255, 255, 0.25);
      --glass-border: rgba(255, 255, 255, 0.18);
    }

    /* Dark Mode (Diwali Theme) */
    body.dark-mode {
      --primary: #ff9800;
      --primary-light: #ffb74d;
      --primary-dark: #f57c00;
      --secondary: #ff5722;
      --secondary-light: #ff8a65;
      --secondary-dark: #d84315;
      --accent: #ffc107;
      --accent-light: #ffd54f;
      --bg: #0a0a2a;
      --bg-light: #1a1a3a;
      --text: #ffffff;
      --text-light: #ffcc00;
      --card-bg: #2d2d2d;
      --shadow: 0 8px 20px rgba(255, 152, 0, 0.2);
      --shadow-hover: 0 15px 30px rgba(255, 152, 0, 0.3);
      --gradient: linear-gradient(135deg, #ff9800, #ff5722);
```

```
  --gradient-secondary: linear-gradient(135deg, #ffcc00, #ff9800);
  --glass-bg: rgba(42, 42, 42, 0.25);
  --glass-border: rgba(255, 152, 0, 0.18);
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Poppins', sans-serif;
  background: var(--bg);
  color: var(--text);
  transition: var(--transition);
  overflow-x: hidden;
  position: relative;
}

/* Glassmorphism effect */
.glass {
  background: var(--glass-bg);
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
  border: 1px solid var(--glass-border);
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.1);
}

/* Diwali Theme Background Elements */
.diwali-background {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  pointer-events: none;
  z-index: -1;
  opacity: 0;
  transition: opacity 0.5s ease;
}

body.dark-mode .diwali-background {
  opacity: 0.6;
}

.candle {
  position: absolute;
  width: 20px;
  height: 40px;
  background: linear-gradient(to bottom, #ff9800, #ff5722);
  border-radius: 10px 10px 0 0;
  box-shadow: 0 0 10px #ff9800;
  animation: flicker 2s infinite alternate;
}
```

```
  position: relative;
  overflow: hidden;
  min-height: 100vh;
  display: flex;
  align-items: center;
  padding: 0;
}

.hero-slider {
  position: relative;
  width: 100%;
  height: 100vh;
  overflow: hidden;
}

.hero-slide {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  opacity: 0;
  transition: opacity 1s ease-in-out;
  background-size: cover;
  background-position: center;
}

.hero-slide.active {
  opacity: 1;
}

.hero-slide::after {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: linear-gradient(135deg, rgba(0,0,0,0.4) 0%, rgba(0,0,0,0.2) 100%);
}

.hero-content {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  text-align: center;
  color: white;
  z-index: 2;
  width: 90%;
  max-width: 1200px;
  padding: 2rem;
}
```

```
    transform: translateY(-5px);
    box-shadow: 0 10px 20px rgba(255, 87, 34, 0.4);

itle {
    font-size: 2.5rem;
    margin-bottom: 1rem;
    font-family: 'Montserrat', sans-serif;
  }

  .offers-subtitle {
    font-size: 1.2rem;
    margin-bottom: 3rem;
    opacity: 0.9;
  }

  .offer-card {
    background: rgba(255, 255, 255, 0.1);
    backdrop-filter: blur(10px);
    border-radius: var(--border-radius);
    padding: 2rem;
    margin-bottom: 2rem;
    text-align: left;
    display: flex;
    align-items: center;
    gap: 2rem;
  }

  .offer-icon {
    font-size: 3rem;
    flex-shrink: 0;
  }

  .offer-content h3 {
    font-size: 1.5rem;
    margin-bottom: 0.5rem;
  }

  .offer-content p {
    opacity: 0.9;
  }

  /* Testimonials */
  .testimonials-section {
    padding: 6rem 2rem;
    background: var(--bg-light);
  }

  .testimonials-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 2rem;
    max-width: 1200px;
    margin: 0 auto;
```

```
  }

body.dark-mode footer {
  background: var(--secondary-dark);
}

.footer-container {
  max-width: 1200px;
  margin: 0 auto;
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 3rem;
}

.footer-column h3 {
  font-size: 1.3rem;
  margin-bottom: 1.5rem;
  position: relative;
  display: inline-block;
}

.footer-column h3::after {
  content: '';
  position: absolute;
  bottom: -8px;
  left: 0;
  width: 40px;
  height: 3px;
  background: var(--secondary);
}

.footer-links {
  list-style: none;
}

.footer-links li {
  margin-bottom: 0.8rem;
}

.footer-links a {
  color: rgba(255,255,255,0.8);
  text-decoration: none;
  transition: var(--transition);
}

.footer-links a:hover {
  color: white;
  padding-left: 5px;
}

.social-icons {
  display: flex;
  gap: 1rem;
  margin-top: 1.5rem;
}
```

```
.social-icons a {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 40px;

  border-radius: 12px;
  box-shadow: var(--shadow-hover);
  padding: 15px;
  z-index: 100;
  transition: var(--transition);
}

.user-dropdown p {
  margin: 0 0 12px 0;
  font-weight: 600;
  color: var(--text);
  font-size: 0.95rem;
}

.user-dropdown button {
  width: 100%;
  padding: 10px;
  border: none;
  border-radius: 8px;
  background: var(--primary);
  color: white;
  cursor: pointer;
  font-weight: 600;
  font-size: 0.9rem;
  transition: var(--transition);
}

.user-dropdown button:hover {
  background: var(--primary-dark);
  transform: translateY(-2px);
}

.user-dropdown::before {
  content: '';
  position: absolute;
  top: -8px;
  right: 20px;
  border-width: 8px;
  border-style: solid;
  border-color: transparent transparent var(--card-bg) transparent;
}

/* Store Hours Modal */
.modal {
  display: none;
  position: fixed;
  top: 0;
  left: 0;
```

```
  width: 100%;
  height: 100%;
  background: rgba(0, 0, 0, 0.7);
  z-index: 2000;
  align-items: center;
  justify-content: center;
}

.day {
  font-weight: 600;
}

.time {
  color: var(--text-light);
}

/* Mobile Menu */
.mobile-menu-btn {
  display: none;
  background: none;
  border: none;
  color: var(--text);
  font-size: 1.5rem;
  cursor: pointer;
}

/* Responsive Design */
@media (max-width: 1024px) {
  .hero-title {
    font-size: 3.5rem;
  }
}

@media (max-width: 768px) {
  .nav-container {
    padding: 1rem;
  }

  .nav-links {
    display: none;
    position: absolute;
    top: 100%;
    left: 0;
    width: 100%;
    background: var(--bg-light);
    flex-direction: column;
    padding: 1rem 0;
    box-shadow: var(--shadow);
  }

  body.dark-mode .nav-links {
    background: var(--bg);
  }

  .nav-links.active {
```

```
        display: flex;
      }

      .mobile-menu-btn {
        display: block;
      }
      .feature-card, .category-card, .testimonial-card {
        padding: 1.5rem;
      }

      .offer-card {
        flex-direction: column;
        text-align: center;
        gap: 1rem;
      }
    }
  </style>
</head>
<body>

  <!-- Diwali Theme Background -->
  <div class="diwali-background">
    <div class="candle"></div>
    <div class="candle"></div>
    <div class="candle"></div>
    <div class="candle"></div>
    <div class="candle"></div>
    <div class="candle"></div>
    <div class="firework"></div>
    <div class="firework"></div>
    <div class="firework"></div>
    <div class="firework"></div>
  </div>

  <!-- Store Hours Modal -->
  <div id="store-hours-modal" class="modal">
    <div class="modal-content glass">
      <div class="modal-header">
        <h3 class="modal-title">Store Hours</h3>
        <button class="close-modal">&times;</button>
      </div>
      <ul class="hours-list">
        <li>
          <span class="day">Monday - Saturday</span>
          <span class="time">8:00 AM - 11:00 PM</span>
        </li>
        <li>
          <span class="day">Sunday</span>
          <span class="time">9:00 AM - 11:00 PM</span>
        </li>
      </ul>
    </div>
  </div>

  <!-- Header -->
```

```
<header class="glass">
  <div class="nav-container">
    <div class="logo" onclick="window.location.href='index.html'">
      <i class="fas fa-shopping-basket"></i>
      <span>FreshMart</span>
    </div>

    <!-- Search Bar -->
    <div class="search-container glass">
      <input type="text" id="product-search" placeholder="Search products..." />
      <button id="search-btn"><i class="fas fa-search"></i></button>
    </div>

    <button class="mobile-menu-btn" id="mobile-menu-btn">
      <i class="fas fa-bars"></i>
    </button>

    <ul class="nav-links" id="nav-links">
      <li>
        <button onclick="window.location.href='index.html'">
          <i class="fas fa-home"></i> Home
        </button>
      </li>

      <li>
        <button onclick="window.location.href='products.html'">
          <i class="fas fa-shopping-basket"></i> Products
        </button>
      </li>

      <li>
        <button onclick="window.location.href='about.html'">
          <i class="fas fa-info-circle"></i> About
        </button>
      </li>

      <li>
        <button onclick="window.location.href='contact.html'">
          <i class="fas fa-phone-alt"></i> Contact
        </button>
      </li>



      <li>
        <button onclick="window.location.href='products.html'" class="shop-btn">
          <i class="fas fa-cart-plus"></i> Shop Now
        </button>
      </li>

      <!-- Settings Dropdown -->
      <li>
        <div class="settings-profile">
          <button id="settings-btn">
              <i class="fas fa-cog"></i>
```

```
          </button>
          <div id="settings-dropdown" class="settings-dropdown glass">
            <div class="settings-item" id="user-profile-item">
              <i class="fas fa-user"></i>
              <span id="user-email">User Email</span>
            </div>
</div>
          <div class="stat-label">Happy Customers</div>
        </div>
        <div class="stat">
          <div class="stat-value">1000+</div>
          <div class="stat-label">Products</div>
        </div>
        <div class="stat">
          <div class="stat-value">50+</div>
          <div class="stat-label">Cities</div>
        </div>
      </div>
    </div>

    <div class="slider-controls">
      <div class="slider-dot active" data-slide="0"></div>
      <div class="slider-dot" data-slide="1"></div>
      <div class="slider-dot" data-slide="2"></div>
      <div class="slider-dot" data-slide="3"></div>
    </div>
  </div>

  <div class="floating-elements">
    <div class="float" style="top: 10%; left: 5%;"><i class="fas fa-apple-alt"></i></div>
    <div class="float" style="top: 20%; right: 10%;"><i class="fas fa-carrot"></i></div>
    <div class="float" style="bottom: 15%; left: 8%;"><i class="fas fa-cheese"></i></div>
    <div class="float" style="bottom: 25%; right: 5%;"><i class="fas fa-wine-bottle"></i></div>
    <div class="float" style="top: 40%; left: 15%;"><i class="fas fa-bread-slice"></i></div>
    <div class="float" style="top: 50%; right: 15%;"><i class="fas fa-egg"></i></div>
  </div>
 </section>

 <!-- Features Section -->
 <section class="features-section" id="features">
  <h2 class="section-title scroll-element">Why Choose FreshMart?</h2>
  <p class="section-subtitle scroll-element">We provide the best grocery shopping experience with quality products and excellent service</p>

  <div class="features-grid">
    <div class="feature-card scroll-element glass">
      <div class="feature-icon">
        <i class="fas fa-shipping-fast"></i>
      </div>
      <h3 class="feature-title">Fast Delivery</h3>
      <p class="feature-description">Get your groceries delivered to your doorstep within 2 hours of ordering.</p>
    </div>

    <div class="feature-card scroll-element glass">
```

```
        <div class="feature-icon">
          <i class="fas fa-leaf"></i>
        </div>
        <h3 class="feature-title">Fresh Products</h3>
        <p class="feature-description">We source directly from farmers to ensure the freshest products for
you.</p>
      </div>

      <div class="feature-card scroll-element glass">
        <div class="feature-icon">
          <i class="fas fa-tag"></i>
        </div>
        <h3 class="feature-title">Best Prices</h3>
        <p class="feature-description">Competitive prices with regular discounts and special offers.</p>
      </div>
    </div>
  </section>

  <!-- Categories Section -->
  <section class="categories-section" id="categories">
    <h2 class="section-title scroll-element">Shop by Category</h2>
    <p class="section-subtitle scroll-element">Browse through our wide range of grocery categories</p>

    <div class="categories-grid">
      <div class="category-card scroll-element glass" onclick="navigateToCategory('fruits')">
        <div class="category-image">
          <i class="fas fa-apple-alt"></i>
        </div>
        <div class="category-info">
          <h3 class="category-title">Fruits & Vegetables</h3>
          <p class="category-count">200+ Products</p>
        </div>
      </div>

      <div class="category-card scroll-element glass" onclick="navigateToCategory('dairy')">
        <div class="category-image">
          <i class="fas fa-cheese"></i>
        </div>
        <div class="category-info">
          <h3 class="category-title">Dairy & Eggs</h3>
          <p class="category-count">150+ Products</p>
        </div>
      </div>

      <div class="category-card scroll-element glass" onclick="navigateToCategory('bakery')">
        <div class="category-image">
          <i class="fas fa-bread-slice"></i>
        </div>
        <div class="category-info">
          <h3 class="category-title">Bakery</h3>
          <p class="category-count">100+ Products</p>
        </div>
      </div>

      <div class="category-card scroll-element glass" onclick="navigateToCategory('pantry')">
```

```
      <div class="category-image">
        <i class="fas fa-wine-bottle"></i>
      </div>
      <div class="category-info">
        <h3 class="category-title">Pantry</h3>
        <p class="category-count">300+ Products</p>
      </div>
    </div>
  </div>
</section>

<!-- Special Offers -->
<section class="offers-section" id="offers">
  <div class="offers-container">
    <h2 class="offers-title scroll-element">Special Offers</h2>
    <p class="offers-subtitle scroll-element">Don't miss out on these amazing deals</p>

    <div class="offer-card scroll-element">
      <div class="offer-icon">
        <i class="fas fa-percentage"></i>
      </div>
      <div class="offer-content">
        <h3>20% Off on First Order</h3>
        <p>Use code WELCOME20 to get 20% off on your first order above ₹500</p>
      </div>
    </div>

    <div class="offer-card scroll-element">
      <div class="offer-icon">
        <i class="fas fa-truck"></i>
      </div>
      <div class="offer-content">
        <h3>Free Delivery</h3>
        <p>Free delivery on all orders above ₹1000. No code required.</p>
      </div>
    </div>
  </div>
</section>

<!-- Testimonials -->
<section class="testimonials-section" id="testimonials">
  <h2 class="section-title scroll-element">What Our Customers Say</h2>
  <p class="section-subtitle scroll-element">Read reviews from our happy customers</p>

  <div class="testimonials-grid">
    <div class="testimonial-card scroll-element glass">
      <p class="testimonial-text">"FreshMart has transformed my grocery shopping experience. The quality of products is exceptional and delivery is always on time!"</p>
      <div class="testimonial-author">
        <div class="author-avatar">RS</div>
        <div class="author-info">
          <h4>Rahul Sharma</h4>
          <p>Regular Customer</p>
        </div>
      </div>
```

```
      </div>

      <div class="testimonial-card scroll-element glass">
        <p class="testimonial-text">"I love how fresh the vegetables and fruits are. The prices are reasonable and the delivery service is excellent. Highly recommended!"</p>
        <div class="testimonial-author">
      <div class="footer-column">
        <h3>Contact Info</h3>
        <ul class="footer-links">
         <li style="list-style: none;">
  <a
href="https://www.google.com/maps?q=Vidyanagar,+Opposite+BalaBharathi+School,+Kotamandal,+Tirupati+District,+Andhra+Pradesh,+India"
     target="_blank"
     style="color: inherit; text-decoration: none; transition: color 0.3s ease;"
     onmouseover="this.style.color='#ffb400'"
     onmouseout="this.style.color='inherit'">
    <i class="fas fa-map-marker-alt"></i> Vidyanagar, Opposite BalaBharathi School, India
  </a>
</li>

         <li style="list-style: none;">
  <a href="tel:+916309872249"
     style="color: inherit; text-decoration: none; transition: color 0.3s ease;"
     onmouseover="this.style.color='#ffb400'"
     onmouseout="this.style.color='inherit'">
    <i class="fas fa-phone"></i> +91 6309872249
  </a>
</li>

         <li style="list-style: none;">
  <a href="mailto:infofreshmart4@gmail.com"
     style="color: inherit; text-decoration: none; transition: color 0.3s ease;"
     onmouseover="this.style.color='#ffb400'"
     onmouseout="this.style.color='inherit'">
    <i class="fas fa-envelope"></i> infofreshmart4@gmail.com
  </a>
</li>

          <li><i class="fas fa-clock"></i> Mon-Sat: 8am-10pm, Sun: 9am-8pm</li>
        </ul>
      </div>
    </div>

    <div class="copyright">
      &copy; 2023 FreshMart Grocery Store. All rights reserved.
    </div>
  </footer>

  <script>
    // Light/Dark Mode Toggle
    document.addEventListener('DOMContentLoaded', function() {
      const themeToggleItem = document.getElementById('theme-toggle-item');
      const themeText = document.getElementById('theme-text');
      const themeIcon = themeToggleItem.querySelector('i');
```

```
  const body = document.body;

  // Check for saved theme preference or use OS preference
  const prefersDarkScheme = window.matchMedia('(prefers-color-scheme: dark)');
  const currentTheme = localStorage.getItem('theme');

  if (currentTheme === 'dark' || (!currentTheme && prefersDarkScheme.matches)) {
    body.classList.add('dark-mode');
    themeText.textContent = 'Light Mode';
    themeIcon.className = 'fas fa-sun';
  } else {
    body.classList.remove('dark-mode');
    themeText.textContent = 'Diwali Mode';
    themeIcon.className = 'fas fa-moon';
  }

  // Toggle theme on button click
  themeToggleItem.addEventListener('click', function() {
    if (body.classList.contains('dark-mode')) {
      body.classList.remove('dark-mode');
      localStorage.setItem('theme', 'light');
      themeText.textContent = 'Diwali Mode';
      themeIcon.className = 'fas fa-moon';
    } else {
      body.classList.add('dark-mode');
      localStorage.setItem('theme', 'dark');
      themeText.textContent = 'Light Mode';
      themeIcon.className = 'fas fa-sun';
    }
  });
});

// Header scroll effect
window.addEventListener('scroll', function() {
  const header = document.querySelector('header');
  if (window.scrollY > 50) {
    header.style.boxShadow = '0 8px 25px rgba(0,0,0,0.1)';
    header.style.background = 'rgba(255, 255, 255, 0.98)';
    if(document.body.classList.contains('dark-mode')) {
      header.style.background = 'rgba(34, 34, 34, 0.98)';
    }
  } else {
    header.style.boxShadow = '0 8px 20px rgba(0,0,0,0.1)';
    header.style.background = 'rgba(255, 255, 255, 0.95)';
    if(document.body.classList.contains('dark-mode')) {
      header.style.background = 'rgba(30, 30, 30, 0.95)';
    }
  }
});

// Mobile menu toggle
const mobileMenuBtn = document.getElementById('mobile-menu-btn');
const navLinks = document.getElementById('nav-links');

mobileMenuBtn.addEventListener('click', () => {
```

```
  navLinks.classList.toggle('active');
});

// Close mobile menu when clicking outside
document.addEventListener('click', (e) => {
  if (!mobileMenuBtn.contains(e.target) && !navLinks.contains(e.target)) {

// Search functionality
const searchInput = document.getElementById("product-search");
const searchBtn = document.getElementById("search-btn");

// Function to search product
function searchProduct() {
  const query = searchInput.value.trim().toLowerCase();
  if (!query) return;

  // Go to products page
  const productsPage = "products.html";

  // Open products page first
  window.location.href = `${productsPage}?search=${encodeURIComponent(query)}`;
}

// Event listeners
searchBtn.addEventListener("click", searchProduct);
searchInput.addEventListener("keypress", function(e) {
  if (e.key === "Enter") searchProduct();
});

// Settings Dropdown
const settingsBtn = document.getElementById('settings-btn');
const settingsDropdown = document.getElementById('settings-dropdown');
const userEmail = document.getElementById('user-email');
const logoutItem = document.getElementById('logout-item');
const storeHoursItem = document.getElementById('store-hours-item');
const storeHoursModal = document.getElementById('store-hours-modal');
const closeModal = document.querySelector('.close-modal');

// Get logged-in user from localStorage
const storedUser = localStorage.getItem('customer');
if (storedUser) {
    const user = JSON.parse(storedUser);
    userEmail.textContent = user.email;
} else {
    // If no user, redirect to login page
    window.location.href = 'login.html';
}

// Toggle settings dropdown
settingsBtn.addEventListener('click', (e) => {
    e.stopPropagation();
    settingsDropdown.classList.toggle('active');
});

// Close settings dropdown when clicking outside
```

```
window.addEventListener('click', () => {
    settingsDropdown.classList.remove('active');
});

// Logout button
logoutItem.addEventListener('click', () => {
    // Remove user from localStorage
    localStorage.removeItem('customer');
    // Redirect to login page
    window.location.href = 'login.html';
});

// Store hours modal
storeHoursItem.addEventListener('click', () => {
    storeHoursModal.style.display = 'flex';
});

// Close modal
closeModal.addEventListener('click', () => {
    storeHoursModal.style.display = 'none';
});

// Close modal when clicking outside
window.addEventListener('click', (e) => {
    if (e.target === storeHoursModal) {
        storeHoursModal.style.display = 'none';
    }
});

// Category navigation
function navigateToCategory(category) {
  window.location.href = `products.html?category=${category}`;
}

// Scroll animations
const scrollElements = document.querySelectorAll('.scroll-element');

const elementInView = (el, dividend = 1) => {
  const elementTop = el.getBoundingClientRect().top;
  return (
    elementTop <= (window.innerHeight || document.documentElement.clientHeight) / dividend
  );
};

const displayScrollElement = (element) => {
  element.classList.add('animate');
};

const hideScrollElement = (element) => {
  element.classList.remove('animate');
};

const handleScrollAnimation = () => {
  scrollElements.forEach((el) => {
    if (elementInView(el, 1.2)) {
```

```
        displayScrollElement(el);
      } else {
        hideScrollElement(el);
      }
    });
  };

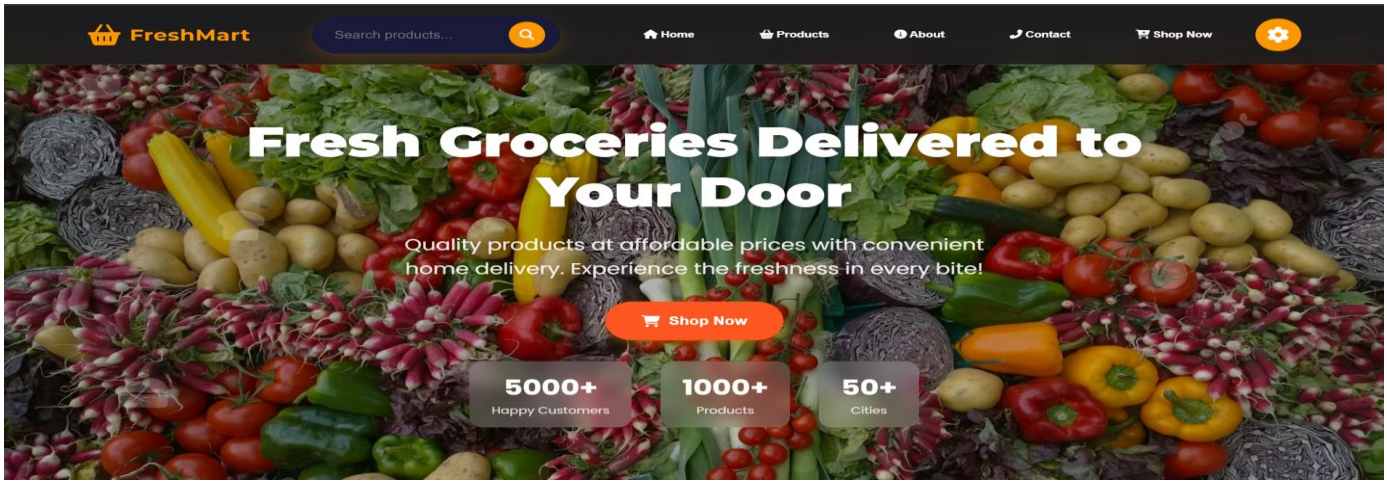
  // Initialize scroll animations
  window.addEventListener('scroll', () => {
    handleScrollAnimation();
  });

  // Initial check for elements in view on page load
  handleScrollAnimation();
</script>

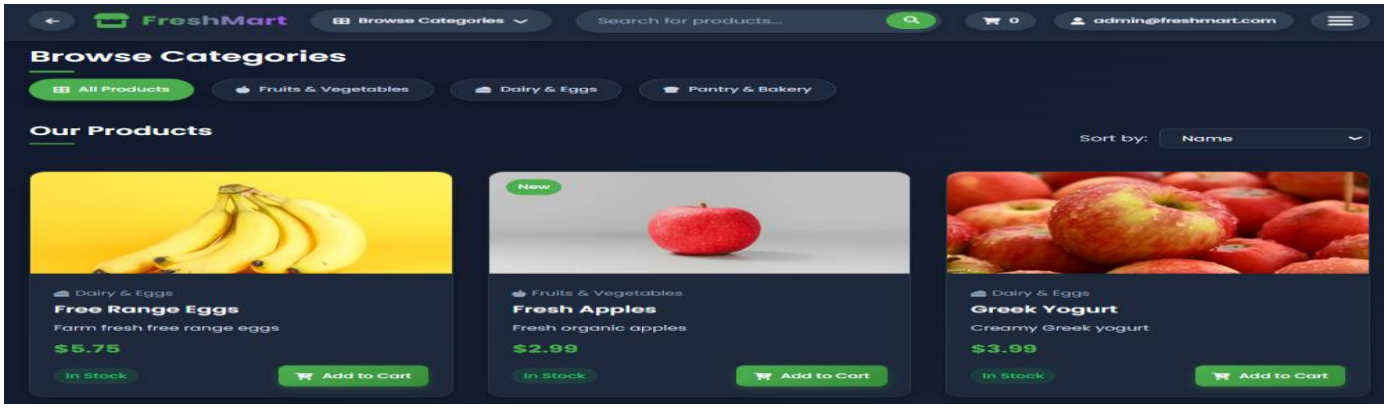
</body>
</html>
```

OUTPUT :

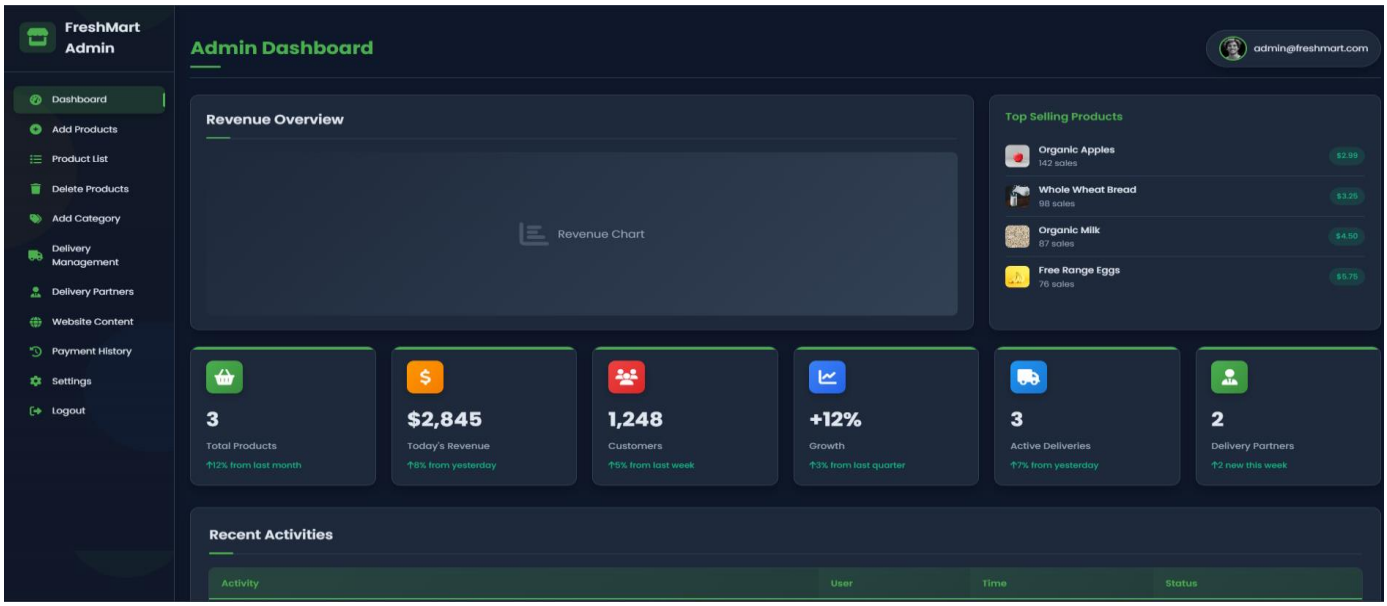
INDEX PAGE :



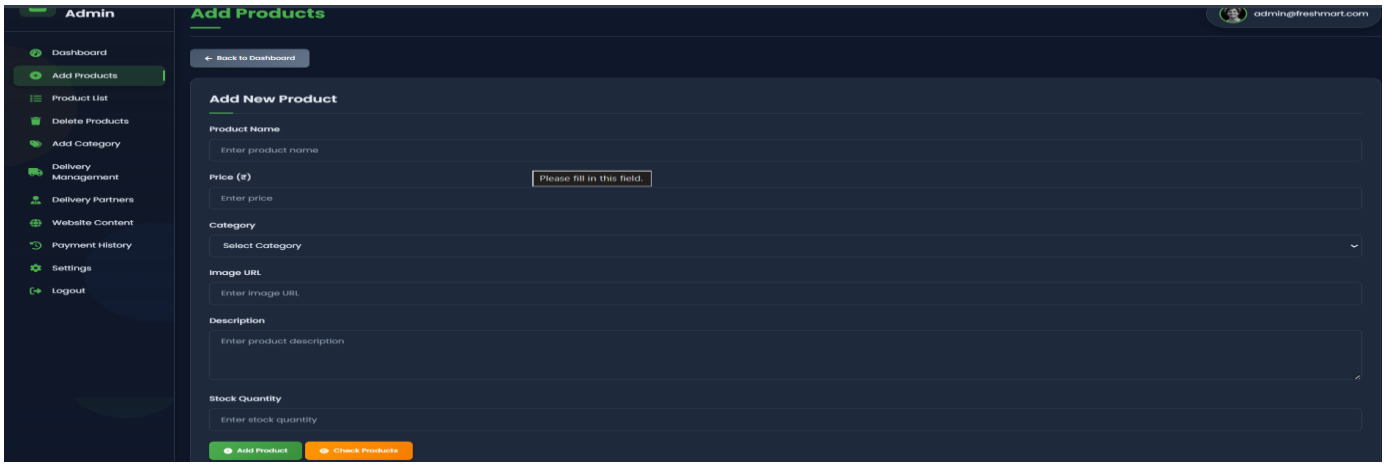
PRODUCTS PAGE :



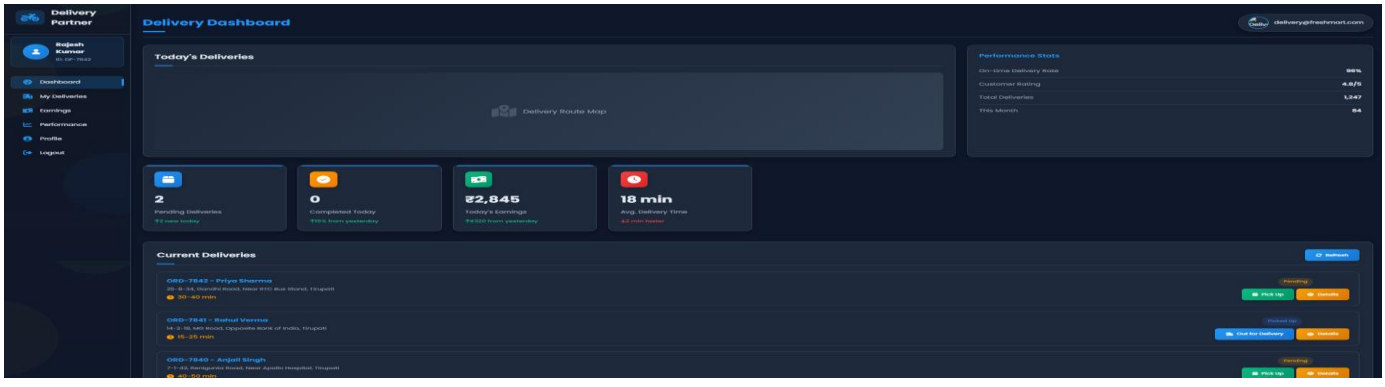
ADMIN DASHBOARD PAGE :



ADD PRODUCTS PAGE :



DELIVERY DASHBOARD PAGE :



Result: Successfully developed a responsive Fresh Mart web application with an optimized layout, fast-loading visuals, and user-friendly shopping experience.

10.2 PORTFOLIO WEB APPLICATION

Problem Statement: Design and develop a personal portfolio website to effectively showcase Nikhil's skills, projects, and professional profile. The website should feature clearly structured sections such as About Me, Skills, Projects, and Contact for easy navigation. It must be visually appealing, clean, and fully responsive across different devices. The goal is to create a strong online presence that highlights achievements and enables seamless communication with potential recruiters or collaborators.

Abstract

The **Personal Portfolio** website developed by Nikhil is designed as a professional and interactive platform to showcase his skills, projects, and overall profile in a structured and visually appealing manner. It acts as an online resume where visitors can easily understand his background, technical expertise, and achievements. The website is built using HTML for structuring content and CSS for styling, ensuring a clean layout and smooth user experience. Each section is thoughtfully organized to present information clearly, helping recruiters or collaborators quickly access relevant details.

The layout The portfolio includes key sections such as About Me, Skills, Projects, and Contact, allowing a complete overview of Nikhil's capabilities and work. The design maintains consistency in fonts, colors, and spacing to create a professional look, while responsive design techniques ensure the site works seamlessly across desktops, tablets, and mobile devices. By combining simplicity with functionality, the project effectively highlights his strengths and provides an easy way for visitors to connect with him. will consist of multiple sections such as an introduction (About Me), a skills section showcasing technical abilities, a projects section with brief descriptions and links, and a contact section for visitors to reach out. Special attention will be given to consistency in design, font usage, spacing, and responsiveness across different screen sizes.

Source Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Portfolio</title>
    <link rel="stylesheet" href="style.css">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">
    <script src="https://kit.fontawesome.com/a076d05399.js"
crossorigin="anonymous"></script>
</head>
<body>

    <header>
        <div class="logo">My Portfolio</div>
        <nav>
            <ul>
                <li><a href="#about">About</a></li>
                <li><a href="#projects">Projects</a></li>
                <li><a href="#contact">Contact</a></li>
            </ul>
        </nav>
    </header>

    <section class="hero">
        <h1>Hello, I'm <span>STEVE ROGERS</span></h1>
        <p>I'm a Web Developer passionate about building beautiful websites.</p>
        <a href="#projects" class="btn">View Projects</a>
    </section>

    <section id="about">
        <h2>About Me</h2>
        <p>I am a front-end web developer with experience in HTML, CSS, and JavaScript.</p>
    </section>

    <section id="projects">
        <h2>Projects</h2>
        <div class="project-list">
            <div class="project">
                <img src="project1.jpg" alt="Project 1">
                <h3>Project 1</h3>
                <p>Details about your project.</p>
```

```
            </div>
            <div class="project">
                <img src="project2.jpg" alt="Project 2">
                <h3>Project 2</h3>
                <p>Details about your project.</p>
            </div>
        </div>
    </section>

    <section id="contact">
        <h2>Contact Me</h2>
        <p>Feel free to reach out via social media.</p>
        <div class="social-icons">
            <a href="https://github.com/yourprofile" target="_blank"><i class="fa-brands fa-
github"></i></a>
            <a href="https://linkedin.com/in/yourprofile" target="_blank"><i class="fa-brands fa-
linkedin"></i></a>
            <a href="https://twitter.com/yourprofile" target="_blank"><i class="fa-brands fa-
twitter"></i></a>
        </div>

    </section>

    <footer>
        <p>© 2025 STEVE ROGERS. All rights reserved.</p>
    </footer>

</body>
</html>
/* Global Styles */
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4;
    color: #333;
}

/* Header */
header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background: #333;
    color: white;
```

```
    padding: 15px 20px;
}

header .logo {
    font-size: 24px;
    font-weight: bold;
}

nav ul {
    list-style: none;
    padding: 0;
    margin: 0;
    display: flex;
}

nav ul li {
    margin: 0 15px;
}

nav ul li a {
    color: white;
    text-decoration: none;
}

/* Hero Section */
.hero {
    text-align: center;
    padding: 100px 20px;
    background: #007bff;
    color: white;
}

.hero h1 span {
    color: #ffdd57;
}

.hero .btn {
    display: inline-block;
    margin-top: 10px;
    padding: 10px 20px;
    background: #ffdd57;
    color: #333;
    text-decoration: none;
    font-weight: bold;
    border-radius: 5px
```

```
}

        /* Sections */
        section {
            text-align: center;
            padding: 50px 20px;
        }

        .project-list {
            display: flex;
            justify-content: center;
            gap: 20px;
            flex-wrap: wrap;
        }

        .project {
            background: white;
            padding: 15px;
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0,0,0,0.1);
            width: 300px;
            text-align: center;
        }

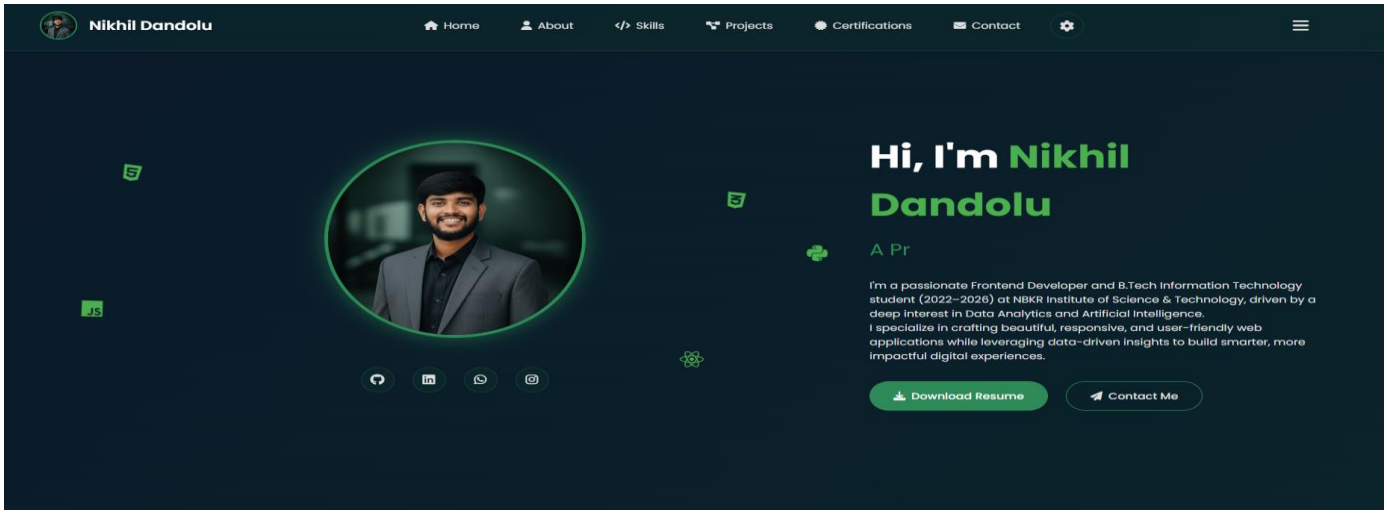
        .project img {
            width: 100%;
            border-radius: 8px;
        }

        /* Social Media Icons */
        .social-icons {
            display: flex;
            justify-content: center;
            gap: 15px;
            margin-top: 20px;
        }

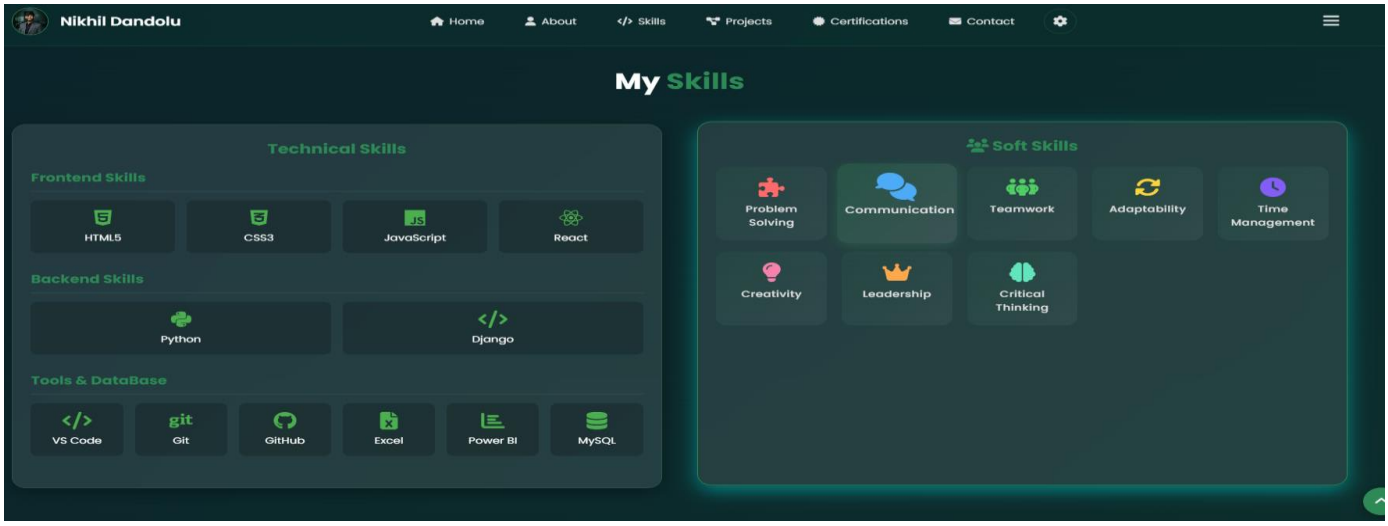
        .social-icons a {
            font-size: 30px;
            color: #333;
            text-decoration: none;
            transition: color 0.3s ease;
        }

        .social-icons a:hover }
```

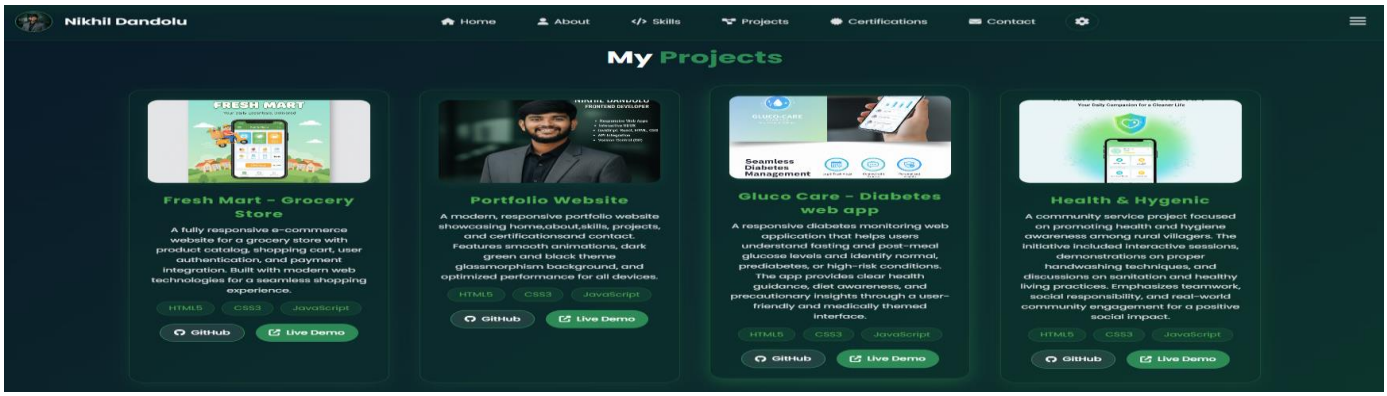
OUTPUT :



SKILLS PAGE :



PROJECT PAGE :



Result: Successfully created a personal portfolio website showcasing skills and projects professionally.

10.3 TICKET BILLING WEB APPLICATION :

Problem Statement: Design and develop a billing web application to manage orders, generate bills, and streamline the billing process for a food service system. The application should provide a clean interface for adding items, calculating totals, and displaying transaction details efficiently. It must be responsive, user-friendly, and capable of improving accuracy and speed in daily billing operations.

Abstract

The Morning Bite Billing Web Application project is designed to create an efficient and user-friendly platform for managing billing operations in a food service environment. This application allows users to add items, calculate totals, and generate accurate bills in real time, simplifying the overall transaction process. The interface is designed to be clean and interactive, enabling quick navigation and smooth order handling, which helps improve both speed and accuracy in daily billing activities.

The front-end of the application is developed using modern web technologies like HTML, CSS, and JavaScript to ensure a responsive and visually appealing design. The system focuses on providing a seamless user experience across different devices, making it accessible on both desktops and mobile screens. By combining structured design with dynamic functionality, the project demonstrates practical implementation of web development concepts, helping users efficiently manage billing tasks while showcasing essential skills such as UI design, data handling, and real-time calculations. front-end will be developed using ReactJS for a dynamic user interface, while JavaScript will handle API requests and logic. Node.js may be used for backend operations or server-side functionalities if needed. This project combines real-time data handling with responsive design, enabling users to stay informed about weather conditions anytime, anywhere. Through this project, learners will gain experience working with APIs, managing state in React, and handling asynchronous operations, all of which are essential skills in modern web development**.**

Source Code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta
      name="description"
      content="Web site created using create-react-app"
    />
    <link rel="apple-touch-icon" href="logo192.png" />
    <link rel="stylesheet" href="%PUBLIC_URL%/app.css" />

    <!--
      manifest.json provides metadata used when your web app is installed on a
      user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-manifest/
    -->
    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
    <!--
      Notice the use of %PUBLIC_URL% in the tags above.
      It will be replaced with the URL of the `public` folder during the build.
      Only files inside the `public` folder can be referenced from the HTML.

      Unlike "/favicon.ico" or "favicon.ico", "%PUBLIC_URL%/favicon.ico" will
      work correctly both with client-side routing and a non-root public URL.
      Learn how to configure a non-root public URL by running `npm run build`.
    -->
    <title>React App</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>
    <!--
      This HTML file is a template.
      If you open it directly in the browser, you will see an empty page.

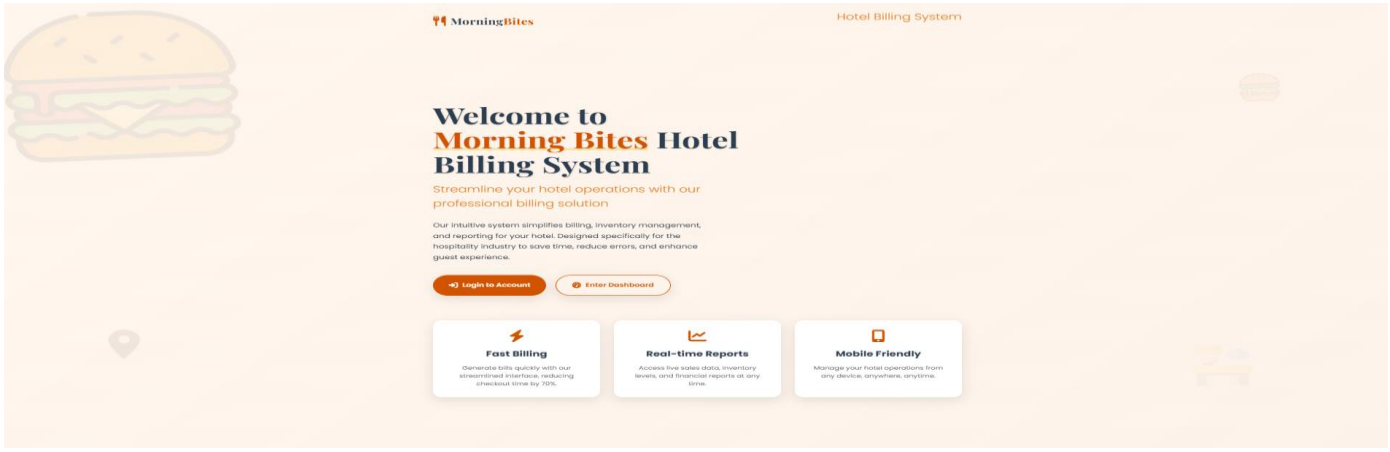
      You can add webfonts, meta tags, or analytics to this file.
      The build step will place the bundled scripts into the <body> tag.

      To begin the development, run `npm start`or `yarn start`.
      To create a production bundle, use `npm run build` or `yarn build`.
    -->
```

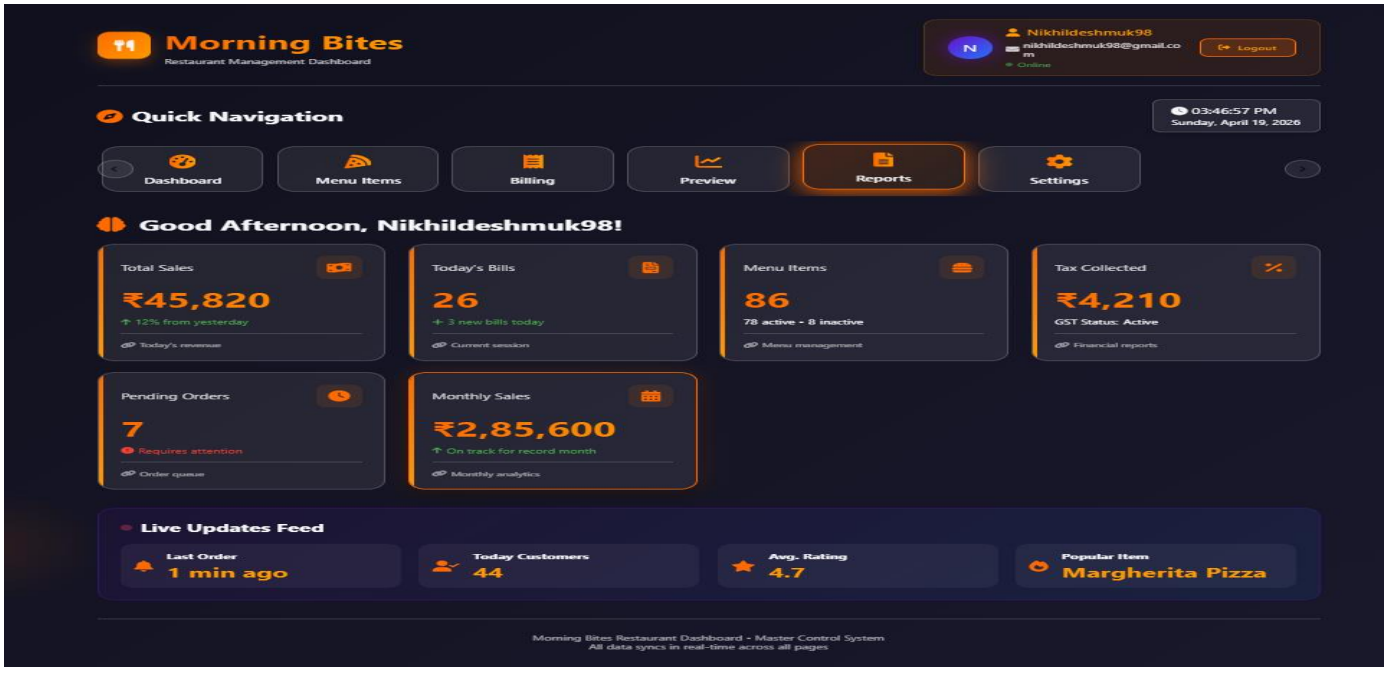
```
  </body>
</html>
{
  "short_name": "React App",
  "name": "Create React App Sample",
  "icons": [
    {
      "src": "favicon.ico",
      "sizes": "64x64 32x32 24x24 16x16",
      "type": "image/x-icon"
    },
    {
      "src": "logo192.png",
      "type": "image/png",
      "sizes": "192x192"
    },
    {
      "src": "logo512.png",
      "type": "image/png",
      "sizes": "512x512"
    }
  ],
  "start_url": ".",
  "display": "standalone",
  "theme_color": "#000000",
  "background_color": "#ffffff"
}{
  "name": "weatherapp",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
  "axios": "^0.19.2",
    "bootstrap": "^4.4.1",
    "react": "^16.13.1",
    "react-bootstrap": "^1.0.0",
    "react-dom": "^16.13.1",
    "react-router-dom": "^5.1.2",
    "react-scripts": "1.1.5"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test --env=jsdom",
    "eject": "react-scripts eject"
  }
```

Output:

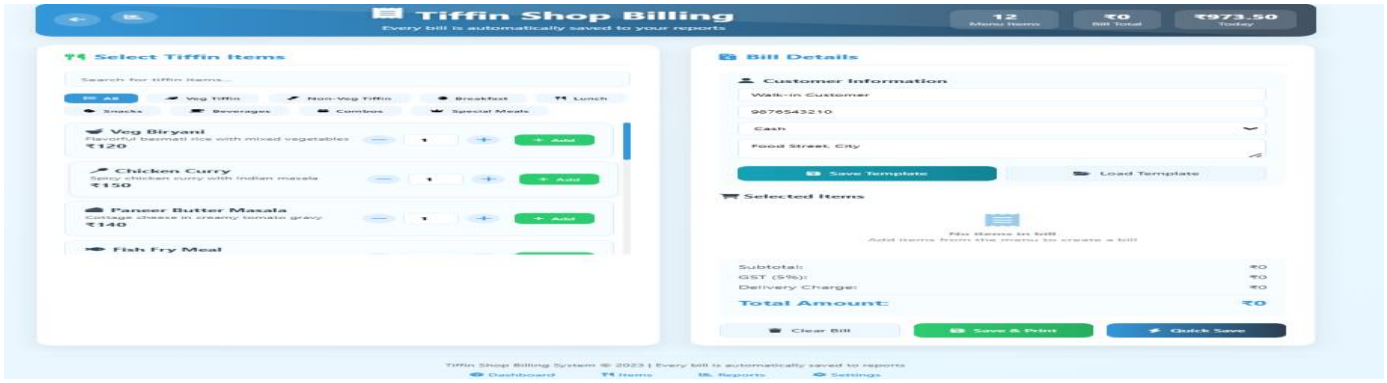
INDEX PAGE :



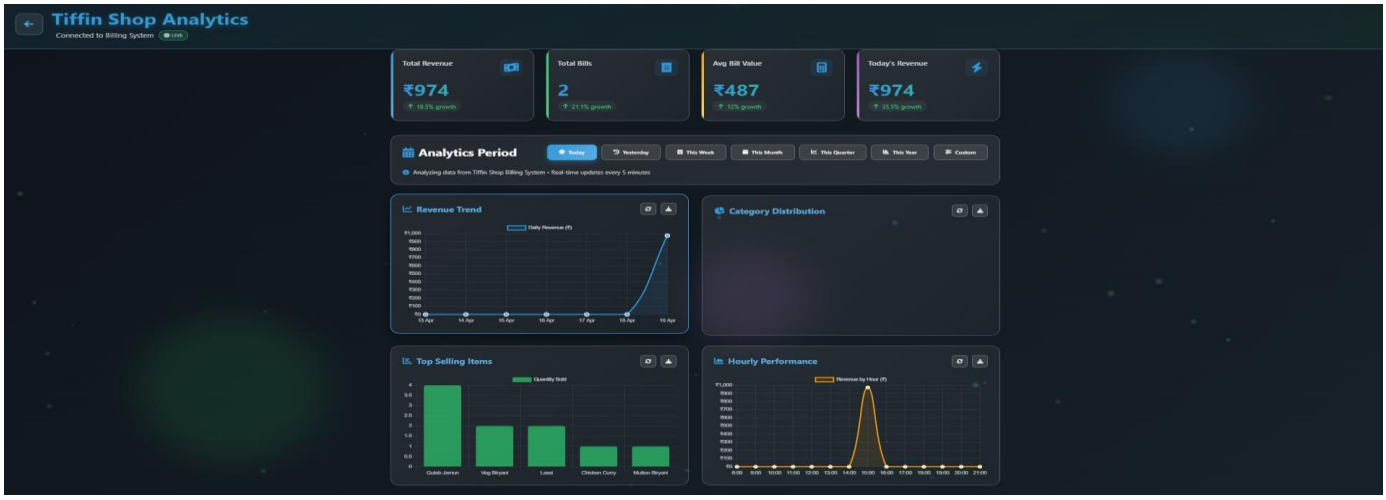
USER DASHBOARD PAGE :



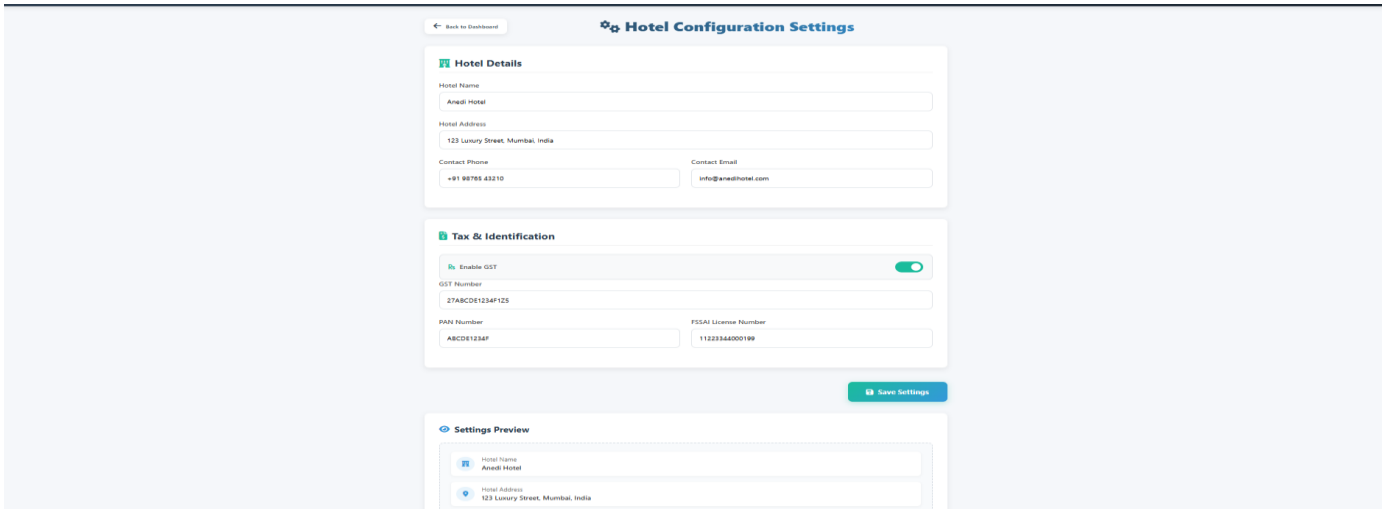
BILLING PAGE :



ANALYTICS DASHBOARD PAGE :



SETTINGS PAGE :



Result: Successfully developed a Morning Bite Billing Web Application that enables efficient order management, accurate bill generation, and a smooth, user-friendly billing experience.

11. CERTIFICATE

11.1 Offer Letter



Slash Mark

OFFER LETTER

Dear Nikhil Dandolu,
Intern ID: SMI82853
N.B.K.R Institute of Science and Technology (NBKRIST)

Congratulations!

We are delighted to present you with an offer for the position of Full Stack Web Development Internship commencing from December 23, 2025 to April 23, 2026.

As an intern, you will have the opportunity to gain valuable experience. As a temporary employee, please be aware that you won't receive the same benefits as our regular staff.

Kindly adhere to our company's policies, including those related to conduct, safety, and confidentiality. We have every confidence that your internship with us will prove to be fulfilling and we extend our best wishes for success in this promising opportunity.

We look forward to welcoming you to our team and witnessing your growth and contributions firsthand.

P Abhishek
Human Resources (HR)
Slash Mark IT Solutions (OPC) Pvt Ltd
info@slashmark.in

MINISTRY OF CORPORATE AFFAIRS
GOVERNMENT OF INDIA

#startupindia

11.2 Certificate



Slash Mark

#startupindia

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that

Nikhil Dandolu

has successfully completed the **Full Stack Web Development Internship** at Slash Mark IT Solutions (OPC) Pvt. Ltd., an ISO 9001:2015 Certified and DPIIT-recognized Organization based in Karimnagar, Telangana, India,

during the period of **December 23, 2025 to April 23, 2026**.

This certificate is issued in recognition of the successful completion of the internship.

P. Abhishek
Human Resources (HR)
Slash Mark IT Solutions (OPC) Pvt. Ltd.
hr@slashmark.in

Intern ID : SMI82853
Credits Awarded: 3
Internship Mode : Virtual

12. CONCLUSION

Through this course, I've gained a solid understanding of both the fundamental and advanced concepts in web development. I've learned the core building blocks like HTML, CSS, and JavaScript, and also explored more advanced tools and frameworks, including ReactJS, which have enhanced my ability to create dynamic and responsive websites.

The course also introduced me to essential backend development concepts with Node.js and API integration, expanding my skills to full-stack development. Additionally, I've learned the importance of UX/UI design principles to create user-friendly interfaces and image optimization techniques to improve website performance.

The hands-on projects throughout the course have given me the opportunity to apply what I've learned in real-world scenarios, and I feel confident in my ability to build both frontend and backend applications. This course has laid a strong foundation, and I'm excited to continue growing and tackling new challenges in web development.